

Práctica 2

Introducción a la microprogramación

GRADO EN INGENIERÍA INFORMÁTICA - Grupo 80

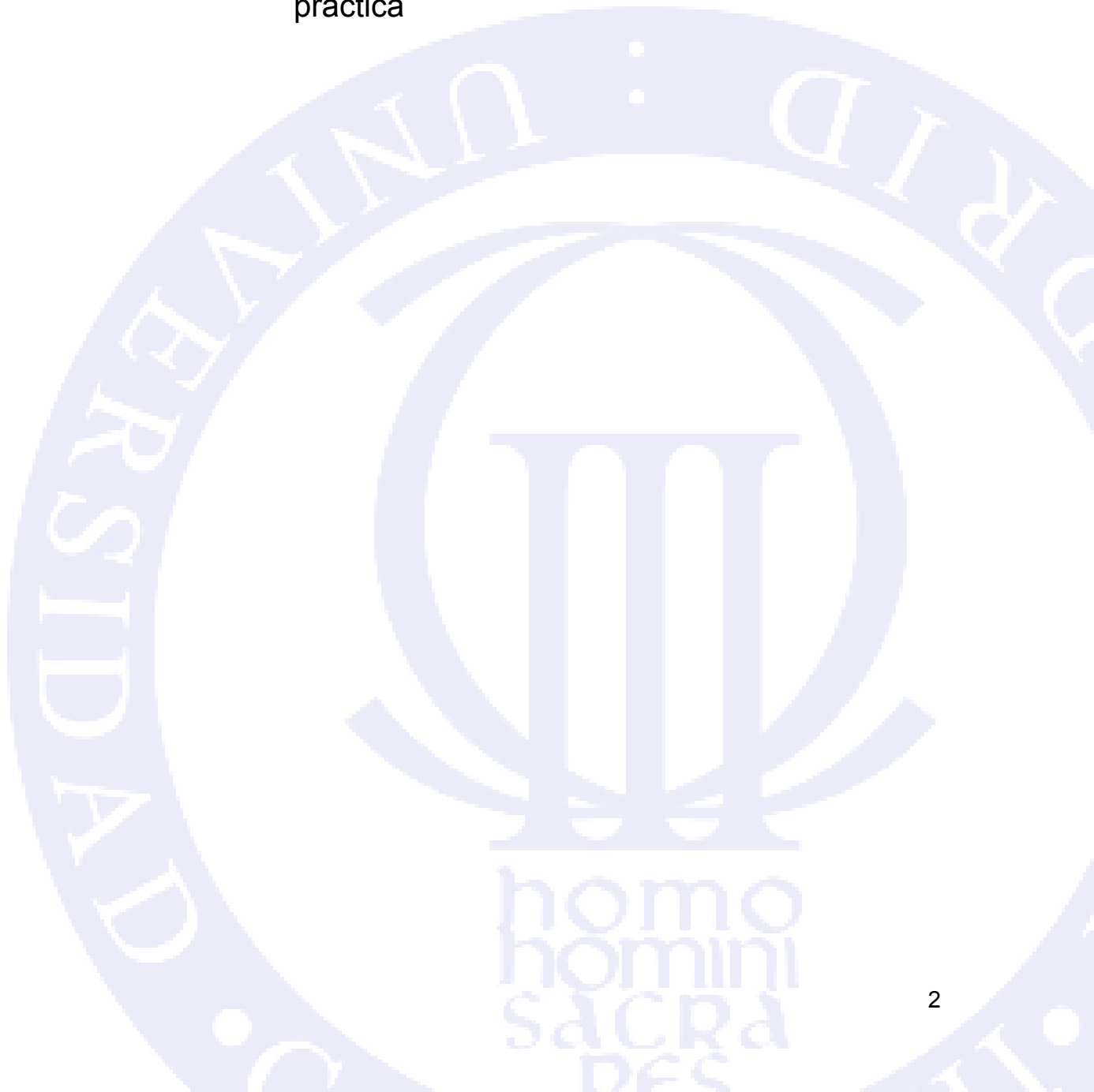
Autores:

Álvaro García Piqueras - 100495760
100495760@alumnos.uc3m.es

Arturo Hernanz Carrasco-Muñoz - 100495850
100495850@alumnos.uc3m.es

ÍNDICE DE CONTENIDOS:

1. Ejercicio 1
 - 1.1. Solución aplicada al ejercicio
2. Ejercicio 2
 - 2.1. Solución aplicada al ejercicio
3. Conclusiones, problemas encontrados y estimación del tiempo empleado en la práctica
 - 3.1. Conclusión
 - 3.2. Problemas encontrados
 - 3.3. Estimación del tiempo empleado en la práctica



1. Ejercicio 1

1.1. Solución aplicada al ejercicio

Instrucciones	Operaciones elementales	Señales de control	Decisiones de diseño
la r1 u32	C1: MAR $\leftarrow$ PC C2: MBR $\leftarrow$ MP[MAR] C3: PC $\leftarrow$ PC+4, BR[r1] $\leftarrow$ MBR	C1: (T2, C0), C2: (TA, R, BW=11, M1=1, C1=1), C3: (M2, C2, T1, LC=1, MR=0, SELC=10101, A0=1, B=1, C=0)	No hay grandes decisiones de diseño más allá de seguir el esquema proporcionado para el comportamiento de cada instrucción.
sc r1, r2, (r3)	C1: MAR $\leftarrow$ BR[r3] C2: MBR $\leftarrow$ BR[r1] C3: MP[MAR] $\leftarrow$ MBR C4: MAR $\leftarrow$ BR[r3+4] C5: MBR $\leftarrow$ BR[r2] C6: MP[MAR] $\leftarrow$ MBR	C1: (MR=0, SELA=01011, T9=1, C0=1), C2: (MR=0, SELA=10101, T9=1, M1=0, C1=1), C3: (BW=11, TA=1, TD=1, W=1), C4: (MR=0, SELA=01011, MA=0, MB=10, SELCOP=01010, MC=1, T6=1, C0=1), C5: (MR=0, SELA=10000, T9=1, M1=0, C1=1), C6: (BW=11, TA=1, TD=1, W=1, A0=1, B=1, C=0)	No hay grandes decisiones de diseño más allá de seguir el esquema proporcionado para el comportamiento de cada instrucción.
lc r1 r2 (r3)	C1: MAR $\leftarrow$ BR[r3] C2: MBR $\leftarrow$ MP[MAR] C3: BR[r1] $\leftarrow$ MBR C4: MAR $\leftarrow$ BR[r3+4] C5: MBR $\leftarrow$ MP[MAR] C6: MAR $\leftarrow$ BR[r2]	C1: (MR=0, SELA=01011, T9=1, C0=1), C2: (TA=1, R=1, BW=11, M1=1, C1=1), C3: (T1=1, LC=1, MR=0, SELC=10101), C4: (MR=0, SELA=01011, MA=0, MB=10, SELCOP=01010, MC=1, T6=1, C0=1), C5: (TA=1, R=1, BW=11, M1=1, C1=1), C6: (T1=1, LC=1, MR=0, SELC=10000, A0=1, B=1, C=0),	No hay grandes decisiones de diseño más allá de seguir el esquema proporcionado para el comportamiento de cada instrucción.
addc r1 r2 r3 r4	C1: ALU $\leftarrow$ BR[r1]+BR[r3];BR[r1] $\leftarrow$ ALU C2: ALU $\leftarrow$ BR[r2]+BR[r4];BR[r2] $\leftarrow$ ALU	C1: (MR=0, SELA=10101, SELB=01011, MA=0, MB=00, MC=1, SELCOP=01010, T6=1,	No hay grandes decisiones de diseño más allá de seguir el esquema

		LC=1, SELC=10101, SelP=11, M7=1, C7=1), C2: (MR=0, SELA=10000, SELB=00110, MA=0, MB=00, MC=1, SELCOP=01010, T6=1, LC=1, SELC=10000, SelP=11, M7=1, C7=1, A0=1, B=1, C=0),	proporcionado para el comportamiento de cada instrucción.
mulc r1 r2 r3 r4	C1: RT1←BR[r1]*BR[r3] C2: RT2←BR[r2]*BR[r4] C3: RT3← RT1 - RT2 C4: RT1←BR[r1]*BR[r4] C5: RT2←BR[r2]*BR[r3] C6: BR[r1]←RT3 C7: BR[r2]← RT1 + RT2	C1: (MR=0, SELA=10101, SELB=01011, SELCOP=1100, MA=0, MB=00, MC=1, T6=1, C4=1, SelP=11, M7=1, C7=1), C2: (MR=0, SELA=10000, SELB=00110, SELCOP=1100, MA=0, MB=00, MC=1, T6=1, C5=1, SelP=11, M7=1, C7=1), C3: (MA=1, MB=01, MC=1, SELCOP=01011, C6=1, SelP=11, M7=1, C7=1), C4: (MR=0, SELA=10101, SELB=00110, SELCOP=1100, MA=0, MB=00, MC=1, T6=1, C4=1, SelP=11, M7=1, C7=1), C5: (MR=0, SELA=10000, SELB=01011, SELCOP=1100, MA=0, MB=00, MC=1, T6=1, C5=1, SelP=11, M7=1, C7=1), C6: (T7=1, LC=1, SELC=10101), C7: (MA=1, MB=01, MC=1, SELCOP=01010, T6=1, LC=1, SELC=10000, SelP=11, M7=1, C7=1, A0=1, B=1, C=0)	No hay grandes decisiones de diseño más allá de seguir el esquema proporcionado para el comportamiento de cada instrucción.
beqc r1 r2 r3 r4 s6	C1: RT2 ← SR C2: RA =BR[r1]; RB = BR[r3]; RA - RB; SR ← FLAGS C3: MUXA ← MADDR + 1 (si z = 0) C4: RA =BR[r2]; RB = BR[r4]; RA - RB; SR ← FLAGS C5: MUXA ← MADDR + 1 (si z = 0) C6: RT2 ← SR C7: RT1 ← PC C8: RT2 ← offset C9: RT1 + RT2 bck2ftch – C1: SR ← RT2 bck2ftch – C2: Llamada al fetch	C1: (T8, C5) C2: (SELA=10101, SELB=01011, MC=1, SELCOP=1011, SELP=11, M7, C7) C3: (A0=0, B=1, C=110, MADDR=bck2ftch) C4: (SELA=10000, SELB=00110, MC=1, SELCOP=1011, SELP=11, M7, C7) C5: (A0=0, B=1, C=110, MADDR=bck2ftch)	Para comparar que dos registros son iguales tuvimos que ver que la resta de ambos fuera 0.

		C6: (T5, M7=0, C7) C7: (T2, C4) C8: (SE=1, OFFSET=0, SIZE=00110, T3, C5) C9: (MA=1, MB=1, MC=1, SELCOP=1010, T6, C2, A0=1, B=1, C=0) bck2ftch – C1: (T5, M7=0, C7) bck2ftch – C2: (A0=1, B=1, C=0)	
call u20	C1: BR[ra]←PC C2: PC←SELEC[U20]	C1: (T2=1, LC=1, MR=1, SELC=00001), C2: (SIZE=10100, T3=1, M2=0, C2=1, A0=1, B=1, C=0)	No hay grandes decisiones de diseño más allá de seguir el esquema proporcionado para el comportamiento de cada instrucción.
ret	C1: PC ← BR[ra]	C1: (MR=1, SELA=00001, T9=1, M2=0, C2=1, A0=1, B=1, C=0)	No hay grandes decisiones de diseño más allá de seguir el esquema proporcionado para el comportamiento de cada instrucción.
hcf	C1: PC←BR[zero];SR←BR[zero]	C1: (MR=1, SELA=0, T9=1, M2=0, C2=1, M7=0, C7=1, A0=1, B=1, C=0)	No hay grandes decisiones de diseño más allá de seguir el esquema proporcionado para el comportamiento de cada instrucción.

2. Ejercicio 2

2.1. Solución aplicada al ejercicio:

	Ciclos de reloj sin extensión	Ciclos de reloj con la extensión	Mejora (%)
A == B	131	83	36.64%
A != B	93	74	20.43%

Comparación juego de instrucciones:

- **la reg, u32:** Es una de las pocas instrucciones que no presenta grandes diferencias entre la extensión y la instrucción original. Como gran ventaja presenta la capacidad de guardar 2 palabras en dos direcciones de memoria seguidas en una única instrucción. La instrucción tiene más complejidad que la original y se realiza en más ciclos de reloj.
- **sc reg1, reg2, (reg3):** Esta instrucción guarda dos direcciones de memoria seguidas en dos registros, su gran ventaja es esta debido a que para llevar a cabo este proceso a través de la instrucción original necesitas utilizarla dos veces ya que esta guarda la dirección de memoria de un registro en otro. Su clara desventaja es que únicamente puede guardar dos direcciones de memoria seguidas a diferencia de la instrucción original que puede guardar la dirección sumada a un valor cualquiera.
- **lc reg1, reg2, (reg3):** Inversa a la anterior esta instrucción guarda el valor de dos registros distintos en dos posiciones de memoria seguidas. Esta es su gran ventaja ya que hace en una instrucción lo que en la instrucción original requeriría dos. Su desventaja es que los datos se guardan en dos posiciones de memoria seguidas y en la instrucción original puedes guardarlo en la dirección sumada a un valor cualquiera.
- **addc reg1, reg2, reg3, reg4:** Esta instrucción se encarga de hacer una suma doble entre registros, en una única instrucción es capaz de realizar dos sumas aunque no es funcional en el caso que solo quieras realizar una única suma, ya que siempre hace dos operaciones.
- **mulc reg1, reg2, reg3, reg4:** Esta operación está orientada a la multiplicación de complejos, en ese caso es una gran ventaja ya que realiza en una única instrucción toda la operación y a través de las instrucciones generales necesitas la suma, la multiplicación y la resta. Su gran desventaja es que está limitada sólo a la multiplicación de complejos y solo es funcional en ese caso.
- **beqc reg1, reg2, reg3, reg4, s6:** Esta instrucción se encarga de comprobar si el valor de los registros es igual para saltar a la dirección dada. Es una instrucción funcional en los números complejos y si quieres realizar dobles comprobaciones ya que es capaz de hacerlas en una única instrucción mientras que con la instrucción original la doble comprobación es más complicada. Su gran desventaja es que siempre realiza dos comprobaciones y en caso de que no lo necesites tiene una mayor complejidad que la necesaria.
- **call u20:** Esta instrucción es igual en la instrucción original y en la extensión, se encarga de guardar la dirección en la que te encuentras y saltar a otra. Es muy necesaria en cualquier programa.
- **ret:** Esta instrucción se encarga de volver a la dirección guardada en el registro BR[ra]. Es una instrucción necesaria en los programas y muy similar a la instrucción de las instrucciones originales.
- **hcf:** Esta instrucción se encarga de terminar el programa, tiene una gran ventaja ya que en cualquier momento puedes finalizar el programa.

Podemos concluir que la extensión merece la pena en el caso de que uses números complejos ya que las instrucciones están orientados a ellos y en un programa en el que no necesites este tipo de número no usaras prácticamente ninguna de las instrucciones, además que algunas de las instrucciones ya están en la versión sin extensión o presentan diferencias mínimas y no sería necesario implementarlas. Las pruebas realizadas

demuestran que, en situaciones que involucren números complejos, la extensión puede mejorar significativamente los ciclos de reloj. Esto se traduce en un rendimiento más rápido y eficiente en comparación con la versión estándar del programa. Las operaciones con números complejos, al ser intrínsecamente más complejas, se benefician enormemente de la optimización proporcionada por la extensión. En resumen, la utilidad de la extensión se hace evidente cuando se trata de trabajar con números complejos. Si el programa en cuestión no requiere el uso frecuente de estos números, la implementación de la extensión puede resultar innecesaria y no justificará la mejora de rendimiento observada en las pruebas.

3. Conclusiones, problemas encontrados y estimación del tiempo empleado en la práctica

3.1 Conclusión

Para la realización de la práctica hemos tenido que familiarizarnos con el funcionamiento de un microprocesador y sus distintas funcionalidades. A lo largo de este trabajo, hemos explorado conceptos esenciales, desde la creación de microinstrucciones hasta la implementación de un conjunto de instrucciones eficientes y personalizadas.

Hemos visto como la microprogramación ha demostrado ser una herramienta valiosa para abordar desafíos en la arquitectura de computadores permitiendo una mayor flexibilidad y la capacidad de personalizar un conjunto de instrucciones según los requisitos pedidos.

Además, hemos examinado ejemplos prácticos de la aplicación de la microprogramación en la mejora del rendimiento de procesadores y la optimización del ciclo de instrucciones. En resumen, este trabajo ha proporcionado una visión integral de la microprogramación, destacando su relevancia y aplicaciones prácticas.

3.2 Problemas encontrados

El principal problema que nos hemos encontrado ha sido la optimización del ciclo de instrucciones ya que encontrar el equilibrio entre el número de ciclos y de instrucción y su complejidad ha sido complicado. La optimización del ciclo de instrucciones es crucial para mejorar el rendimiento del procesador.

Otro problema que nos encontramos fue la implementación de la instrucción beqc, debido a que nuestro diseño establece que la resta de los registros debe ser 0 para la comprobación posterior pero al iniciar la instrucción no sabíamos que desarrollo seguir. Otro problema de esta instrucción fue que interpretamos S6 como un "inm" y nos daba un error en el número de bits hasta que entendimos que es un "address-rel".

3.3 Estimación del tiempo empleado en la práctica

Este proyecto implicó el desarrollo e implementación de 9 instrucciones de microprogramación con sus respectivas pruebas y la realización de la memoria. Aproximadamente la realización de la práctica al completo es de 12 horas.

